

ASSIGNMENT 2

Design of a Vector Embedding for Capability Composition

Formal Representations, Compatibility, and Compositional Reasoning

Application Domain: Online Shopping / E-Commerce

Embedding Approach: Problem-Specific Vector Representation

Technical Report

This report presents the design, implementation, and experimental evaluation of a problem-specific vector representation for formally specified states, goals, and executable capabilities.

Niranjana

B.Tech Computer Science and Engineering

APJ Abdul Kalam Technological University (KTU)

1. Problem Definition

The objective of this assignment is to design a problem-specific vector representation for formally specified states, goals, and executable capabilities. The representation should preserve useful relationships between capabilities so that compatibility, composition, goal relevance, and operational properties can be analysed in vector space.

The application domain is online shopping. An e-commerce workflow contains capabilities such as adding products to a cart, checking inventory, creating an order, making payment, generating an invoice, and sending notifications. These capabilities have explicit preconditions, effects, inputs, outputs, resources, costs, and reliability values, making the domain suitable for evaluating functional composition.

The work focuses on the embedding and composition representation rather than on redesigning a planning algorithm. A vector should not merely give every capability a unique identifier; it should contain information useful for reasoning about relationships between capabilities.

2. Design Requirements

The representation was designed to satisfy the following requirements:

- **Capability identity:** different capabilities have distinguishable representations.
- **State awareness:** state and goal information is representable as vectors.
- **Precondition-effect compatibility:** an effect produced by one capability can satisfy a precondition of another.
- **Input-output compatibility:** capability inputs and outputs are part of the representation.
- **Similarity versus composability:** high vector similarity alone must not imply that two capabilities can be executed consecutively.
- **Composition:** vectors of compatible capabilities can be combined into a composite capability representation.
- **Goal relevance:** capabilities useful for the selected goal can be distinguished from irrelevant ones.
- **Operational properties:** cost, reliability, and availability are represented.

3. Related Embedding Approaches

Word2Vec is relevant as background motivation because it shows how discrete objects can be represented as vectors in a continuous space and how relationships can be reflected through vector similarity. However, word embeddings alone are not sufficient here, because capability composition depends on structured functional relationships such as preconditions, effects, inputs, outputs, and resources.

Therefore, this project uses a custom, problem-specific representation rather than a general-purpose pretrained embedding model. The representation is intentionally interpretable: tokens derived from structured capability and state descriptions contribute to the vector, while operational attributes are appended as numerical features.

4. Proposed Representation

The application is described using the formal model $\mathbf{A} = (\mathbf{S}, \mathbf{C}, \mathbf{S}_I, \mathbf{G}, \mathbf{R}, \mathbf{K})$, where $\mathbf{S}$ is the state space, $\mathbf{C}$ the set of capabilities, $\mathbf{S}_I$ the initial state, $\mathbf{G}$ the goal, $\mathbf{R}$ the set of resources, and $\mathbf{K}$ the set of constraints.

4.1 Initial State

- User.authenticated = True
- User.role = CUSTOMER
- Cart.exists = True
- Cart.item_count = 3
- Order.exists = False
- Order.status = NONE
- Payment.status = NOT_STARTED
- Inventory.available = True
- Invoice.exists = False
- Notification.sent = False

4.2 Goal State

- Order.exists = True
- Payment.status = SUCCESS
- Invoice.exists = True
- Notification.sent = True

4.3 Capability Set

Capability	Type	Main effect / purpose
AddToCart	FUNCTION	Adds a product and increases cart contents
CheckInventory	DATABASE	Checks product availability
CreateOrder	API	Creates an order from a non-empty cart
MakePayment	SERVICE	Completes payment for a created order
GenerateInvoice	FUNCTION	Creates an invoice after successful payment
SendNotification	MESSAGE	Sends a notification after successful payment
CancelCart	FUNCTION	Cancels an existing cart when no order exists
UpdateProfile	GUI	Updates the authenticated user's profile

4.4 Vector Construction

A vocabulary is built from the structured text describing the capabilities, the initial state, and the goal. Each item is represented by a normalized token-frequency vector. For capabilities, three operational features are appended: cost, reliability, and availability. A capability vector therefore has the form:

$$\phi(C_i) = [\text{text_features}(C_i), \text{cost}(C_i), \text{reliability}(C_i), \text{availability}(C_i)]$$

State and goal vectors are produced by the same text-vector mechanism, without the operational feature block. When a state or goal vector is compared with a capability vector, it is aligned to the capability dimension (the three operational positions are zero-filled).

5. Mathematical Formulation

Let $V = \{v_1, v_2, \dots, v_n\}$ be the vocabulary extracted from the structured descriptions. For an object x , the text representation is the normalized token vector:

$$t(x) = [f_1(x), \dots, f_n(x)] / \|[f_1(x), \dots, f_n(x)]\|$$

where $f_j(x)$ is the frequency of vocabulary token v_j in the description of x . For a capability C_i , the final representation is:

$$\phi(C_i) = [t(C_i), q_i], q_i = [\text{cost}, \text{reliability}, \text{availability}]$$

Because only the text block is unit-normalized, $\phi(C_i)$ itself is not a unit vector; cosine similarity is therefore computed with explicit norms:

$$\text{sim}(x, y) = (\phi(x) \cdot \phi(y)) / (\|\phi(x)\| \|\phi(y)\|)$$

Compatibility is not defined by similarity. For two capabilities C_i and C_j , the transition $C_i \rightarrow C_j$ is **compatible** when the effects of C_i satisfy the state preconditions required by C_j .

Composition of a valid capability chain $C_1 \rightarrow C_2 \rightarrow \dots \rightarrow C_k$ uses the component-wise mean of the capability vectors:

$$\phi(C_1 \rightarrow \dots \rightarrow C_k) = (1/k) \sum \phi(C_i)$$

The mean gives a simple composite representation; the validity of the chain is determined separately by the functional compatibility rules.

6. Capability Composition Model

Each capability is represented using its type, inputs, outputs, preconditions, effects, constraints, resources, cost, reliability, availability, and execution mechanism. The capabilities that take part in the composition experiments are specified below.

Capability	Inputs → Outputs	Preconditions	Effects
CreateOrder	cart_id → order_id	Cart exists; cart item count > 0; inventory available	Order exists; status CREATED; cart locked
MakePayment	order_id, amount → transaction_id	Order exists; status CREATED	Payment status SUCCESS
GenerateInvoice	order_id, transaction_id → invoice_id	Payment status SUCCESS	Invoice exists
SendNotification	user_id, message → notification_id	Payment status SUCCESS	Notification sent
CancelCart	cart_id → status	Cart exists; Order does not exist	Cart cancelled; cart removed

The main composition chain evaluated is **CreateOrder → MakePayment → GenerateInvoice**. It is valid because CreateOrder establishes the created-order state required by MakePayment, and MakePayment establishes the successful payment required by GenerateInvoice.

This chain, named CompletePurchase, achieves three of the four goal conditions (Order.exists, Payment.status = SUCCESS, Invoice.exists). Goal condition Notification.sent = True additionally requires SendNotification, which is compatible after MakePayment. A complete plan from the initial state is therefore CreateOrder → MakePayment → {GenerateInvoice, SendNotification}. CheckInventory is not required, because Inventory.available is already True in the initial state.

An important distinction demonstrated by the experiments is that vector similarity and functional composability are different concepts: two capabilities may have similar representations while still being incompatible because their state conditions do not match.

7. Implementation

The implementation is written in Python. The main program constructs the vocabulary, encodes states, goals, and capabilities, computes cosine similarity, checks compatibility using precondition–effect relationships, performs vector composition, executes the five experiments, and saves the experimental dataset to *dataset.json*.

Main functions implemented:

- `encode_state(state)`
- `encode_goal(goal)`
- `encode_capability(capability)`
- `similarity(x, y)`
- `compatibility(c1, c2)`
- `compose(capabilities)`

The final execution produced an embedding dimension of 64 for the configured dataset and completed all five experiments successfully.

8. Experimental Methodology

Five experiments were conducted according to the assignment requirements.

Experiment	Purpose
1. Capability Compatibility	Test compatible and incompatible capability pairs.
2. Capability Composition	Construct and compare a multi-capability composite vector.
3. Alternative Implementations	Compare API, database, and GUI implementations of the same functionality.
4. Irrelevant Capability	Compare goal similarity for relevant and irrelevant capabilities.
5. Operational Attributes	Demonstrate cost, reliability, and availability features.

9. Results

9.1 Experiment 1: Capability Compatibility

Pair	Compatible?	Similarity
CreateOrder → MakePayment	True	0.733
CreateOrder → CancelCart	False	0.730
MakePayment → GenerateInvoice	True	0.707
MakePayment → SendNotification	True	0.695

CreateOrder → CancelCart has a similarity of 0.730 (higher than MakePayment → GenerateInvoice at 0.707 and MakePayment → SendNotification at 0.695) yet is incompatible: CancelCart requires that no order exists, while CreateOrder creates one. This directly shows that similarity cannot replace functional compatibility.

9.2 Experiment 2: Capability Composition

Relationship	Similarity
Composite vs CreateOrder	0.902
Composite vs MakePayment	0.906
Composite vs GenerateInvoice	0.892

The composite is CompletePurchase = CreateOrder → MakePayment → GenerateInvoice. Its similarity to each component is above 0.89. This is the expected behaviour of a mean of vectors that are already mutually similar, so the result confirms that the composite lies in the region of its components; it does not by itself show that functional information is preserved.

9.3 Experiment 3: Alternative Implementations

Pair	Similarity
CreateOrder_API vs CreateOrder_DATABASE	0.471
CreateOrder_API vs CreateOrder_GUI	0.680
CreateOrder_DATABASE vs CreateOrder_GUI	0.577

Different execution mechanisms for the same functionality are represented in one vector space and keep measurable similarity (0.47–0.68). The spread shows that mechanism-specific tokens, not only the shared functionality, influence the vector.

9.4 Experiment 4: Irrelevant Capability

Capability	Relevance	Goal Similarity
CreateOrder	Relevant	0.115
MakePayment	Relevant	0.258
GenerateInvoice	Relevant	0.277
SendNotification	Relevant	0.236
UpdateProfile	Irrelevant	0.000
CancelCart	Irrelevant	0.000

The relevant capabilities have non-zero goal similarity, while the two irrelevant capabilities have zero similarity, meaning they share no vocabulary tokens with the goal description. Goal similarity here is a token-overlap measure; it separates the two groups but does not rank usefulness exactly (e.g., CreateOrder, which is essential, scores lowest among the relevant capabilities).

9.5 Experiment 5: Operational Attributes

Capability	Cost	Reliability	Availability
AddToCart	0.000	0.990	1.000
CheckInventory	0.000	0.995	1.000
CreateOrder	0.010	0.980	1.000
MakePayment	0.500	0.970	1.000
GenerateInvoice	0.010	0.990	1.000
SendNotification	0.020	0.950	1.000
CancelCart	0.000	0.990	1.000
UpdateProfile	0.000	0.990	1.000

These values are the operational attributes used by the implementation and saved in *dataset.json*. MakePayment is the only high-cost capability (0.5); availability is 1.0 for every capability, so it

does not discriminate between them in this dataset.

10. Analysis

The experiments show that the representation encodes the structured information needed for the selected e-commerce capability-composition problem.

- **Compatibility:** the explicit precondition–effect check correctly separates the compatible and incompatible pairs.
- **Similarity is not composability:** the CreateOrder → CancelCart pair (0.730, incompatible) versus MakePayment → GenerateInvoice (0.707, compatible) shows that similarity must be interpreted separately from functional validity.
- **Composition:** the composite vector stays close to its components (> 0.89), as expected for a mean; chain validity comes from the compatibility rules, not from the vector.
- **Alternative implementations:** API, database, and GUI variants share one vector space and can be compared by cosine similarity.
- **Goal relevance:** relevant capabilities have non-zero goal similarity and the selected irrelevant ones have zero, reflecting token overlap with the goal.
- **Operational information:** cost, reliability, and availability are explicitly part of each capability vector.

Overall, the implementation is a simple and interpretable baseline for vector-based capability composition, designed around the formal structure of the application rather than a generic language embedding.

11. Limitations

- The text representation uses normalized token frequencies and does not capture the deeper semantic relationships of learned neural embeddings.
- Composition is a component-wise mean, so it does not preserve the order of capabilities in a chain.
- The mean is also not meaningful for operational attributes of a composite: total cost should be the sum, and chain reliability the product, of the component values. For CompletePurchase these are cost 0.52 and reliability $0.98 \times 0.97 \times 0.99 \approx 0.941$, whereas the mean vector gives averaged values.
- The appended operational block (e.g., cost 0.5 for MakePayment) is on a different scale from the unit-normalized text block and can influence cosine similarity unevenly.
- Compatibility is based on exact state precondition/effect matching and does not cover complex logical expressions or inferred state transitions.
- Input-output information is included in the capability description, but the compatibility experiments check only state precondition–effect satisfaction; input-output compatibility is therefore not independently tested.
- Operational attributes are represented numerically but not optimized in a multi-objective planner.
- The dataset is small and domain-specific; larger datasets and additional domains are needed to assess generalization.

12. Conclusion

This project designed, implemented, and evaluated a problem-specific vector representation for formally specified states, goals, and executable capabilities in an online-shopping domain. The representation combines structured text features with operational attributes and supports state encoding, goal encoding, capability encoding, similarity measurement, compatibility checking, and capability composition.

The experiments show that compatible capabilities can be identified, incompatible capabilities remain distinguishable even when their vector similarity is high, capability chains can be composed, alternative implementations can be compared, irrelevant capabilities can be separated from goal-related ones, and operational properties can be included in the representation.

The work provides an interpretable baseline for representing capability relationships in vector space. Future improvements include learned embeddings, richer logical compatibility models, order-sensitive composition with sum/product aggregation of cost and reliability, and larger experimental datasets.

Appendix A: Submission Files

- **main.py** – Python implementation of the embedding and experiments.
- **dataset.json** – Generated experimental dataset.
- **README.md** – Project description and instructions for running the implementation.
- **Assignment_2_Final_Report.pdf** – This technical report.